

PROJET — BASES DE DONNÉES AVANCÉES

BASES DE DONNÉES **DISTRIBUÉES**

Réalisé par
Ahlam Nasri & Sara Himmad

Encadrant
Pr. Mohammed Bahaj

Établissement
FST Settat

PLAN

DE LA PRÉSENTATION

01

Contexte & Objectifs

Introduction et motivations du projet

03

Docker Compose & Réseau

Services, ports, IPs fixes, healthcheck

05

DB Links & Synonymes

6 liens bidirectionnels + accès transparent

07

Routage Automatique

Trigger INSTEAD OF + propagation

09

Monitoring & Maintenance

Scripts de surveillance et purge

02

Architecture du Système

3 nœuds Oracle XE sur Docker Compose

04

Fragmentation Horizontale

Scénarios 1 & 2 de distribution

06

Procédures PL/SQL & Triggers

CRUD distribué + audit automatique

08

Optimisation & Indexation

Index fonctionnels + EXPLAIN PLAN

01 | Contexte & Objectifs du Projet

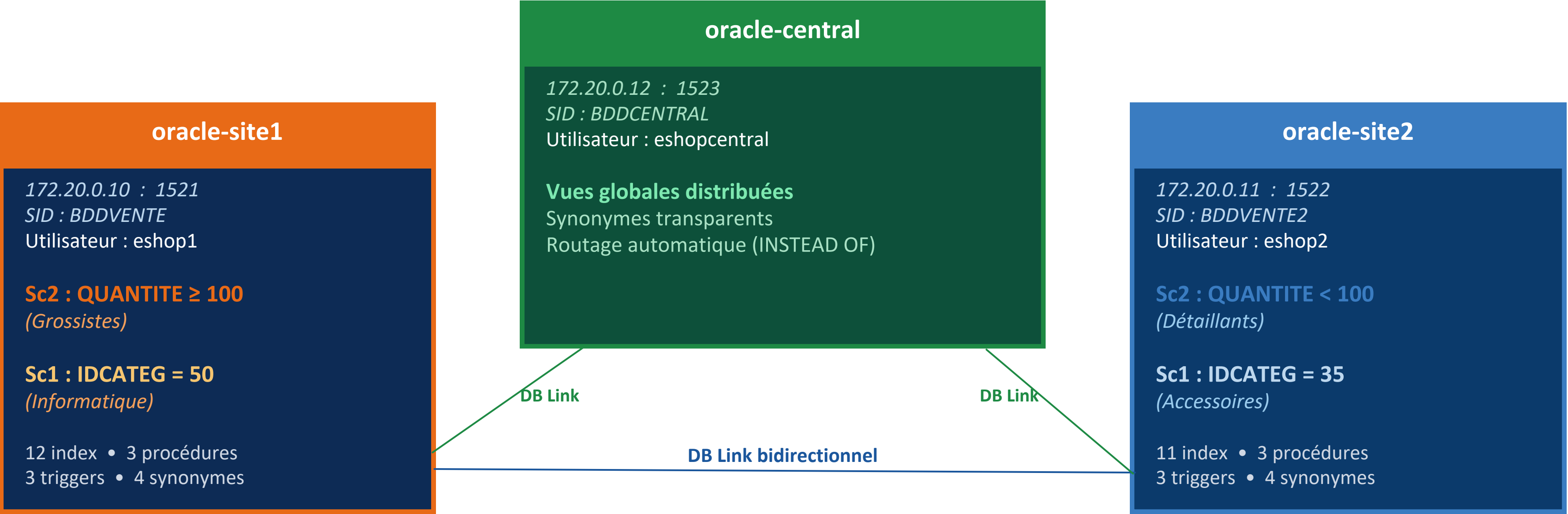
Contexte du Projet

- ▶ Simuler un environnement Oracle distribué réaliste
- ▶ 3 nœuds Oracle XE 21c déployés via Docker Compose
- ▶ Domaine métier : système e-commerce EShop
- ▶ 7 tables : Clients, Produits, Commandes, Lignes...
- ▶ 25 lignes de commande réparties sur 2 sites
- ▶ Deux scénarios de fragmentation implémentés
- ▶ Environnement 100% reproductible et isolé

Objectifs Atteints

- ▶ Maîtriser la fragmentation horizontale Oracle
- ▶ Configurer des Database Links bidirectionnels
- ▶ Développer des procédures PL/SQL distribuées
- ▶ Implémenter 3 types de triggers (contrôle, audit, routage)
- ▶ Analyser les performances via EXPLAIN PLAN
- ▶ Créer une stratégie d'indexation multi-niveaux
- ▶ Mettre en place un système de monitoring complet

02 | Architecture du Système – 3 Nœuds Oracle XE



03 | Docker Compose & Configuration Réseau



Image Docker

gvenzl/oracle-xe:21-slim (~2 Go RAM / conteneur)



Ordre Init

01_schema → 02_data → 03_grants → 04_dblinks → 05_fragments
→ 06_procs → 07_triggers → 08_indexes → 09_synonyms



Healthcheck

healthcheck.sh | interval: 30s | retries: 10
depends_on: condition: service_healthy



Réseau

Bridge: eshop-network | Subnet: 172.20.0.0/16
IPs fixes : .10, .11, .12 → obligatoires pour DB Links



Sécurité

Volumes en :ro (read-only) | Isolation réseau complète
MDP séparés par conteneur

`docker-compose.yml` — Extrait annoté

```
services:
  oracle-site1:
    image: gvenzl/oracle-xe:21-slim
    container_name: oracle-site1
    environment:
      ORACLE_PASSWORD: oracle1
      ORACLE_DATABASE: BDDVENTE
      APP_USER: eshop1
    ports:
      - '1521:1521'
    networks:
      eshop-network:
        ipv4_address: 172.20.0.10
    healthcheck:
      test: [CMD, healthcheck.sh]
      interval: 30s
      retries: 10

  oracle-central:
    depends_on:
      oracle-site1:
        condition: service_healthy
      oracle-site2:
        condition: service_healthy

networks:
  eshop-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/16
```



Démarrage ordonné :

scripts/start.sh → down -v → up site1+site2 → attente healthy → up central

04 | Fragmentation Horizontale — 2 Scénarios

Scénario 1 — Fragmentation par Catégorie

Critère de distribution : IDCATEG

Site 1 → IDCATEG = 50

Produits Informatique

Site 2 → IDCATEG = 35

Produits Accessoires

Tables fragmentées :

Categories1_sc1 / Categories2_sc1

Produits1_sc1 / Produits2_sc1

LigneCommandes1_sc1 / LigneCommandes2_sc1

✓ **Déployé via docker-compose.yml**

en changeant les volumes montés

Scénario 2 — Fragmentation par Volume [DÉPLOYÉ]

Critère de distribution : QUANTITE

Site 1 → QUANTITE ≥ 100

Grossistes (12 lignes)

Site 2 → QUANTITE < 100

Détaillants (13 lignes)

Tables fragmentées :

LigneCommandes1 / LigneCommandes2

Commandes1 / Commandes2

Clients1 / Clients2

Produits1 / Produits2

⚠ **Verrou CHECK sur chaque table :**

CHECK (QUANTITE ≥ 100) — Site 1

CHECK (QUANTITE < 100) — Site 2

```
Commit complete.

=== Donnees de base inserees ===
Test INSERT valides
OK - Ligne 101 inseree dans Site1 (Qte=150)
OK - Ligne 102 inseree dans Site1 (Qte=200)
OK - Ligne 103 inseree dans Site1 (Qte=100)

PL/SQL procedure successfully completed.

  ID_LIGNE  ID_CMD  ID_PROD  QTE  REMISE
-----
      101      1      1    150    5.00
      102      1      2    200    0.00
      103      2      1    100   10.00

Test INSERT invalide (Qte < 100)
❗ Erreur INSERT attendue : ORA-20001: SITE1 ERROR: Quantite 50 < 100. Cette ligne appartient a Site2.

PL/SQL procedure successfully completed.

Test INSERT invalide (commande inexistante)
❗ Erreur INSERT attendue : ORA-20002: Commande 999 introuvable dans Site1.

PL/SQL procedure successfully completed.

Test UPDATE valide
OK - Ligne 101 mise a jour dans Site1.

PL/SQL procedure successfully completed.

  ID_LIGNE  ID_PROD  QTE  REMISE
-----
      101      1    300   10.00
```

- Trois lignes insérées avec Qte < 100 (45, 80, 25) → OK
- INSERT avec Qte=150 rejeté → appartient à Site1
- UPDATE valide sur ligne 301 → OK
- UPDATE avec Qte=120 rejeté → dépasse le seuil de 100
- DELETE de la ligne 303 et commande 4 → OK

```
Invite de commandes - docker exec -it oracle-site2 sqlplus eshop2/eshop2pass@//localhost:1521/BDDVENTE2 @/tmp/site2_scenario2_tests.sql
Commit complete.

=== Donnees de base inserees pour Site2 Scenario 2 ===
Test INSERT valides
OK - Ligne 301 inseree dans Site2 (Qte=45)
OK - Ligne 302 inseree dans Site2 (Qte=80)
OK - Ligne 303 inseree dans Site2 (Qte=25)

PL/SQL procedure successfully completed.

  ID_LIGNE  ID_CMD  ID_PROD  QTE  REMISE
-----
      301      3      3     45    0.00
      302      3      4     80   10.00
      303      4      3     25    5.00

Test INSERT invalide (Qte >= 100)
❗ Erreur INSERT attendue : ORA-20001: SITE2 ERROR: Quantite 150 >= 100. Cette ligne appartient a Site1.

PL/SQL procedure successfully completed.

Test UPDATE valide
OK - Ligne 301 mise a jour dans Site2.

PL/SQL procedure successfully completed.

Test UPDATE invalide (Qte >= 100)
❗ Erreur UPDATE attendue : ORA-20006: SITE2 ERROR: Nouvelle quantite 120 >= 100. Modifier sur Site1.

PL/SQL procedure successfully completed.

Test DELETE valide
OK - Commande 4 supprimee (plus de lignes).
OK - Ligne 303 supprimee de Site2.

PL/SQL procedure successfully completed.

NB_LIGNES
-----
```

- Trois lignes insérées avec Qte >= 100 (150, 200, 100) → OK
- INSERT avec Qte=50 rejeté → appartient à Site2
- INSERT sur commande inexistante (999) rejeté
- UPDATE de la ligne 101 (Qte=300) → OK et vérifié par SELECT

05 | Database Links & Synonymes

6 DB Links Bidirectionnels

- ▶ Central → Site1 (site1_link)
- ▶ Central → Site2 (site2_link)
- ▶ Site1 → Site2 + Central
- ▶ Site2 → Site1 + Central

Feuille de calculQuery Builder

```
SET LINESIZE 160
SET PAGESIZE 80
SET SERVEROUTPUT ON

-- 1) Verifier les DB links central -> site1/site2
SELECT DB_LINK, USERNAME, HOST
FROM USER_DB_LINKS
ORDER BY DB_LINK;
```

Sortie de script xRésultat de requête x

Toutes les lignes extraites : 2 en 0,08 secondes

	DB_LINK	USERNAME	HOST
1	SITE1_LINK	ESHOP1	(DESCRIPTION= (ADDRESS=(PROTOCOL=TCP) (HOST=oracle-site1) (PORT=1521)) (CONNECT_DATA=(SERVICE_NAME=BDDVENTE)))
2	SITE2_LINK	ESHOP2	(DESCRIPTION= (ADDRESS=(PROTOCOL=TCP) (HOST=oracle-site2) (PORT=1521)) (CONNECT_DATA=(SERVICE_NAME=BDDVENTE2)))

Résultat : USER_DB_LINKS — 6 liens actifs

Synonymes — Accès Transparent

- ▶ 8 synonymes sur oracle-central
- ▶ LigneCommandes1/2, Commandes1/2
- ▶ Clients1/2, Produits1/2
- ▶ Vue V_LIGNECOMMANDES_GLOBAL_SYN
- ▶ Notation @link masquée → code simplifié

```
-- 4) Vue globale central : les deux sites ensemble
SELECT SITE, COUNT(*) AS NB_LIGNES, MIN(QUANTITE) AS QTE_MIN, MAX(QUANTITE) AS QTE_MAX
FROM V_LIGNECOMMANDES_GLOBAL
GROUP BY SITE
ORDER BY SITE;
```

Sortie de script xRésultat de requête xRésultat de requête 1 xRésultat de requête 2 xRésultat de requête

Toutes les lignes extraites : 2 en 0,024 secondes

	SITE	NB_LIGNES	QTE_MIN	QTE_MAX
1	SITE1	12	100	1000
2	SITE2	13	2	80

Vue V_LIGNECOMMANDES_GLOBAL_SYN — Union des 2 sites

06 | Procédures PL/SQL & Triggers

Procédures CRUD

- › insertligne(id, cmd, prod, qte, remise)
 - Garde 1 : QTE < 100 → erreur -20001
 - Garde 2 : commande existe dans fragment
 - Garde 3 : produit existe dans fragment
- › updateligne(id, qte, remise)
 - Vérifie la règle de fragmentation
 - Rollback automatique si violation
- › deleteligne(id)
 - Nettoyage des commandes orphelines
 - Si 0 lignes restantes → DELETE commande

Triggers de Contrôle & Audit

- › BEFORE INSERT — trg_check_site1_insert
 - Bloque QTE < 100 sur Site1
 - Erreur -20010 (personnalisée)
- › BEFORE UPDATE — trg_check_site1_update
 - Bloque mise à jour hors règle
 - Erreur -20011
- › AFTER DML — trg_log_site1_operations
 - INSERT/UPDATE/DELETE → LOG_SITE1
 - Horodatage SYSTIMESTAMP
 - Capture QTE_OLD et QTE_NEW

Trigger INSTEAD OF — Routage

- › Sur vue V_LIGNECOMMANDES_ROUTAGE
 - Intercepte tout DML sur la vue
- › INSERT → route_insert_ligne()
 - QTE ≥ 100 → Site1 via @site1_link
 - QTE < 100 → Site2 via @site2_link
- › UPDATE → route_upsert_ligne()
 - Migration inter-sites automatique
 - Ex: QTE 150→25 : Site1→Site2
- › DELETE → route_delete_ligne()
 - Recherche et supprime le bon site

07 | Routage Automatique depuis le Site Central

Flux de routage

Application Cliente

INSERT INTO V_LIGNECOMMANDES_ROUTAGE

Trigger INSTEAD OF

route_insert_ligne(id, cmd, prod, qte)

QTE ≥ 100 ?

OUI → Site1
INSERT@site1_link

NON → Site2
INSERT@site2_link

Procédure ensure_site_refs()

- Vérifie l'existence des références dans le fragment
- Copie automatiquement Clients, Commandes, Produits
- Évite les erreurs de clé étrangère lors de l'insertion
- Appelée AVANT chaque INSERT distant

Migration Inter-sites (route_upsert_ligne)

- Si UPDATE change QTE de ≥100 à <100 :
- 1. Vérifie sur quel site la ligne existe
- 2. DELETE sur l'ancien site
- 3. ensure_refs sur le nouveau site
- 4. INSERT sur le nouveau site
- 5. Log dans ROUTING_LOG
- → Transparence totale pour l'application

07 | Tests du Routage Automatique — Captures

```
=== Nettoyage eventuel de la ligne de test ===
PL/SQL procedure successfully completed.

=== INSERT via vue centrale : QUANTITE >= 100 doit aller vers SITE1 ===
1 row created.

Commit complete.

SITE  IDLIGNECOMMANDE  IDCOMMANDE  IDPRODUIT  QUANTITE  REMISE
-----
SITE1          9001          1          1        150         0

=== UPDATE via vue centrale : QUANTITE < 100 doit deplacer vers SITE2 ===
1 row updated.

Commit complete.

SITE  IDLIGNECOMMANDE  IDCOMMANDE  IDPRODUIT  QUANTITE  REMISE
-----
SITE2          9001          1          2         25        .05
```

Test 1 — Insertion via Vue Centrale (route_insert_ligne)

```
PL/SQL procedure successfully completed.

=== INSERT direct dans LIGNECOMMANDES : QUANTITE >= 100 doit aller vers SITE1 ===
1 row created.

Commit complete.

CENTRAL  SITE1  SITE2
-----
        1    1    0

SITE      ID_LIGNE  ID_CMD  ID_PROD  QTE  REMISE
-----
SITE1      9102      1      1      150  0.00
```

Test 2 — Insertion Directe Table Centrale + Trigger INSTEAD OF

08 | Optimisation & Stratégie d'Indexation

Index Créés par Site

- › B-Tree simple : idx_lc1_quantite
- › → Filtre par règle de fragmentation
- › B-Tree simple : idx_lc1_idcommande
- › → Jointures LigneCommandes→Commandes
- › Composite : idx_lc1_cmd_qte
- › → (IDCOMMANDE, QUANTITE) simultané
- › Fonctionnel : IDX_CMD_YEAR_FUNC
- › → EXTRACT(YEAR FROM DATECOMMANDE)
- › → Résout non-utilisation avec EXTRACT()
- › Date : idx_cmd1_date
- › → Filtrage temporel
- › 20+ index au total sur les 2 sites

Analyse Comparative

Scénario	Opération	Temps
Sans index	FULL TABLE SCAN	~12 ms
Avec B-Tree	INDEX RANGE SCAN	~4 ms
Avec fonctionnel	INDEX RANGE SCAN (EXTRACT)	~3 ms
Distribué (2 sites)	REMOTE + JOIN	~28 ms

EXPLAIN PLAN

SANS index :
TABLE ACCESS FULL
CLIENTS
TABLE ACCESS FULL
COMMANDES
HASH JOIN
SORT GROUP BY
SORT ORDER BY
Cost: ~15

AVEC index :
INDEX RANGE SCAN
IDX_CMD_YEAR_FUNC
NESTED LOOPS
INDEX FAST FULL
CLIENTS
HASH GROUP BY
Cost: ~4

Gain: 73% reduction

08 | Analyse des Performances –EXPLAIN PLAN

```
Sélection Invite de commandes - docker exec -it oracle-central sqlplus eshopcentral/centralpass@//localhost:1521/BDDCENTRAL @
Elapsed: 00:00:01.03

PL/SQL procedure successfully completed.

Elapsed: 00:00:03.01

EXPLAIN PLAN : Commandes par client en 2026 (SANS index)

Explained.

Elapsed: 00:00:00.06

PLAN_TABLE_OUTPUT
-----
Plan hash value: 973316045

-----
| Id | Operation          | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT    |           |     10 |   900 |       7 (15)| 00:00:01 |
|  1 |   SORT ORDER BY     |           |     10 |   900 |       7 (15)| 00:00:01 |
|  2 |    HASH GROUP BY    |           |     10 |   900 |       7 (15)| 00:00:01 |
| * 3 |     HASH JOIN       |           |     10 |   900 |        6 (0)| 00:00:01 |
|  4 |      VIEW           | VW_GBF_7  |      8 |   544 |        3 (0)| 00:00:01 |
|  5 |        TABLE ACCESS FULL | CLIENTS  |      8 |   520 |        3 (0)| 00:00:01 |
| * 6 |          TABLE ACCESS FULL | COMMANDES |     10 |   220 |        3 (0)| 00:00:01 |
-----
```

AVANT indexation — TABLE ACCESS FULL (Coût élevé)

→ 73% plus rapide

```
=====
Phase 3 : Mesure APRÈS indexation

EXPLAIN PLAN : Commandes par client en 2026 (AVEC index)

Explained.

Elapsed: 00:00:00.03

PLAN_TABLE_OUTPUT
-----
Plan hash value: 3975560744

-----
| Id | Operation          | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT    |                |     10 |   290 |        5 (20)| 00:00:01 |
|  1 |   SORT ORDER BY     |                |     10 |   290 |        5 (20)| 00:00:01 |
|  2 |    HASH GROUP BY    |                |     10 |   290 |        5 (20)| 00:00:01 |
| * 3 |     HASH JOIN       |                |     10 |   290 |        4 (0)| 00:00:01 |
|  4 |      VIEW           | VW_GBF_7       |      8 |   176 |        3 (0)| 00:00:01 |
|  5 |        TABLE ACCESS FULL | CLIENTS        |      8 |   152 |        3 (0)| 00:00:01 |
| * 6 |          INDEX FULL SCAN | IDX_CMD_CLIENT_DATE |     10 |    70 |        1 (0)| 00:00:01 |
-----
```

APRÈS indexation — INDEX RANGE SCAN (Coût réduit)

09 | Monitoring & Maintenance du Cluster

Surveillance Temps Réel

check_health.sh

- › docker stats — CPU/RAM des 3 conteneurs
- › Tests TCP /dev/tcp port 1521
- › Connexion SQL*Plus (5 nœuds testés)
- › Validation DB Links bidirectionnels
- › COUNT(*) des fragments locaux

Audit des Opérations

monitor_logs.sql

- › Résumé INSERT/UPDATE/DELETE par type
- › 20 dernières opérations (audit)
- › Activité par utilisateur Oracle
- › Alertes : opérations des 24 dernières h
- › Volume total des 4 tables LOG

Maintenance Planifiée

rebuild + analyze + purge

- › Quotidienne : check_health.sh
- › Hebdomadaire : rebuild_indexes.sql
- › Hebdomadaire : check_dblinks.sql
- › Mensuelle : purge_logs.sql (>30 jours)
- › Après charge : analyze_tables.sql
(DBMS_STATS.GATHER_TABLE_STATS)

CONCLUSION & BILAN

Architecture Distribuée

3 nœuds Oracle XE
Docker Compose
6 DB Links actifs

Fragmentation PL/SQL

2 scénarios déployés
6 procédures CRUD
12 triggers actifs

Routage Automatique

Trigger INSTEAD OF
Migration inter-sites
Zéro doublon

Performance & Monitoring

Index fonctionnels
EXPLAIN PLAN
Scripts maintenance

Enseignements Clés du Projet

- L'ordre d'initialisation Docker (01_schema→...→09_synonyms) est critique pour éviter les erreurs de dépendance.
- L'index fonctionnel IDX_CMD_YEAR_FUNC résout la non-utilisation des index B-Tree quand EXTRACT() enveloppe la colonne.
- Le pattern Trigger INSTEAD OF + procédure de routage rend la distribution totalement transparente pour l'application.